Setting Up the Knowledge Graph Kernel

1 Purpose

This document outlines the steps to create and configure a Jupyter kernel named **Knowledge Graph**, including the installation of all necessary dependencies.

2 Step 1: Create a Conda Environment

1. Open your terminal.
2. Create a new Conda environment named `knowledge-graph`:

```
conda create -n knowledge-graph python=3.9 -y
```

3. Activate the environment:

```
conda activate knowledge-graph
```

3 Step 2: Install Required Packages

Use pip to install the required packages:

```
pip install aiohappyeyeballs aiohttp aiosignal annotated-types anyio appnope
    argon2-cffi attrs Babel backoff beautifulsoup4 bleach blis Bottleneck Brotli
     cached-property catalogue certifi cffi chardet charset-normalizer click
    cloudpathlib colorama comm confection contourpy cycler cymem dataclasses-
    json debugpy decorator deepdiff defusedxml emoji en-core-web-lg en-core-web-
    sm entrypoints et-xmlfile exceptiongroup executing fastjsonschema filelock
    filetype fonttools fqdn frozenlist fsspec greenlet h11 h2 hpack httpcore
    httpx huggingface-hub hyperframe idna importlib_metadata importlib_resources
     ipykernel ipython isoduration jedi Jinja2 joblib json5 jsonpatch jsonpath-
    python jsonpickle jsonpointer jsonschema jsonschema-specifications
    jupyter_client jupyter_core jupyter-events jupyter-lsp jupyter_server
    jupyter_server_terminals jupyterlab jupyterlab_pygments jupyterlab_server
    kiwisolver langchain langchain-community langchain-core langchain-text-
    splitters langcodes langdetect langsmith language_data lxml marisa-trie
    markdown-it-py MarkupSafe marshmallow matplotlib matplotlib-inline mdurl
    mistune mpmath multidict munkres murmurhash mypy-extensions nbclient
    nbconvert nbformat nest_asyncio networkx nltk notebook_shim numexpr numpy
    openpyxl ordered-set orjson overrides packaging pandas pandocfilters parso
    patsy pexpect pickleshare pillow pip pkgutil-resolve-name platformdirs
    preshed prometheus_client prompt_toolkit psutil ptyprocess pure_eval
    pycparser pydantic pydantic_core Pygments pyobjc-core pyobjc-framework-Cocoa
     pyparsing pypdf PySocks python-dateutil python-iso639 python-json-logger
    python-magic pytz pyvis PyYAML pyzmq rapidfuzz referencing regex requests
    requests-toolbelt rfc3339-validator rfc3986-validator rich rpds-py
    safetensors scikit-learn scipy seaborn Send2Trash sentence-transformers
    setuptools shellingham six smart-open sniffio soupsieve spacy spacy-legacy
    spacy-loggers SQLAlchemy srsly stack-data statsmodels sympy tabulate
    tenacity terminado thinc threadpoolctl tinycss2 tokenizers tomli torch
    tornado tqdm traitlets transformers typer types-python-dateutil
    typing_extensions typing-inspect typing-utils tzdata unstructured
```

```
unstructured-client uri-template urllib3 wasabi wcwidth weasel webcolors
webencodings websocket-client wheel wrapt xx-ent-wiki-sm yachalk yarl zipp
zstandard
```

4 Step 3: Register the Kernel with Jupyter

1. Install the IPython kernel package:

```
pip install ipykernel
```

2. Add the kernel to Jupyter:

```
python -m ipykernel install --user --name=knowledge-graph --display-
    name "Knowledge Graph"
```

5 Step 4: Verify the Kernel

Run the following command to list all kernels and verify that **Knowledge Graph** has been added:

```
jupyter kernelspec list
```

6 Step 5: Share the Environment

If sharing with others, export the list of dependencies:

```
pip freeze > requirements_knowledge_graph.txt
```

7 Step 6: Recreate the Environment (for others)

1. Share the `requirements_knowledge_graph.txt` file.
2. The recipient can recreate the environment by running:

```
conda create -n knowledge-graph python=3.9 -y
conda activate knowledge-graph
pip install -r requirements_knowledge_graph.txt
```

Each document provides a standalone guide for setting up its respective kernel. Let me know if you need further refinements!